UNIVERSIDAD CATÓLICA DE SANTA MARÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

LENGUAJES DE PROGRAMACIÓN III

Laboratorio 06 MVC en Java

Modelo Vista Controlador

Actividades 1 a 3 y ejercicios 1 y 2

Estudiante

Gian Piero Khalil Rodríguez Fádel

Repositorio

Crudo27 / Lab6

Arequipa Perú

2026

Índice

1 Objetivos y alcance 3

2 Actividades 4

3 Ejercicios propuestos 11

4 Cuestionario 21

5 Bibliografía 24

6 Anexos 25

El informe presenta la solución acumulativa de pedidos, los dos ejercicios elegidos, la explicación de los archivos y las imágenes de ejecución. Los anexos permiten localizar y ejecutar el código del repositorio.

1 Objetivos y alcance

Objetivos

Implementar el patrón Modelo Vista Controlador en Java separando los datos y las reglas de negocio, la lectura y presentación de información, y la coordinación de las operaciones del usuario. Comprobar esta separación con una aplicación de pedidos y un carrito de compras que incorpora autenticación y reseñas.

Alcance de la práctica

Parte de la guía	Desarrollo incluido
Actividad 1	Agregar y listar pedidos con las clases del ejemplo MVC.
Actividad 2	Tipo de plato, eliminación, actualización, búsqueda por nombre o tipo y conteos.
Actividad 3	Estados, pedidos completos, filtros, pendientes e historial.
Ejercicios 1 y 2	Carrito de compras ampliado con usuarios, reseñas y calificaciones.
Cuestionario	Las 11 preguntas, con citas y bibliografía científica.

Elegí los ejercicios 1 y 2 porque el segundo amplía directamente el carrito del primero. Los ejercicios 3, 4 y 5 quedan fuera de esta entrega. Las actividades se integran en lab6_actividad1 y los ejercicios en lab6_ejercicios1_2.

Criterio de implementación

Conservé la organización del repositorio en modelo, vista y controlador, los nombres en español, las llaves de apertura en una línea aparte para los métodos y el uso de for, ArrayList y TreeMap. La lógica se expresa con condiciones y recorridos directos. Añadí el paquete principal para iniciar cada proyecto.

El código se entrega sin comentarios y la documentación se desarrolla en este informe. Esta decisión cumple la indicación de entrega, aunque la rúbrica incluye comentarios dentro del criterio de documentación. La puntualidad y el trabajo en equipo dependen de la entrega y de la participación real; las pruebas técnicas no permiten acreditar esos aspectos.

Los programas funcionan por consola y guardan los datos en memoria durante una ejecución. La guía no exige persistencia, una interfaz gráfica ni pagos reales. Las imágenes simulan la consola de Eclipse con salidas obtenidas al ejecutar el código, agrupando operaciones de una misma sesión.

2 Actividades

Actividad 1 Registro y listado de pedidos

La aplicación conserva el caso del restaurante planteado en la guía. Main crea el modelo, la vista y el controlador. El menú permite solicitar el nombre y el tipo de plato, registrar el pedido y mostrar los pedidos actuales. ArrayList permite añadir pedidos sin fijar de antemano la cantidad de registros.

La separación de datos, presentación e interacción corresponde al propósito de MVC descrito por Krasner y Pope (1988). En esta implementación de consola, el controlador solicita las entradas a la vista, llama al modelo y entrega el resultado nuevamente a la vista.

Actividad 2 Operaciones adicionales

Se incorporaron la búsqueda por nombre y por tipo, la actualización del nombre de un pedido pendiente, la eliminación y el conteo total y por tipo. La búsqueda compara cadenas completas con equalsIgnoreCase y elimina espacios externos con trim. Una búsqueda sin coincidencias devuelve una lista vacía; la vista informa el resultado sin recibir null.

Cada pedido tiene un ID que se asigna una sola vez. La selección del usuario se hace mediante ese ID y no mediante una posición del ArrayList. Por eso eliminar el pedido 1 no convierte al pedido 2 en otro registro. TreeMap agrupa los tipos sin distinguir mayúsculas y minúsculas y mantiene un orden alfabético al presentar los conteos.

Actividad 3 Estados e historial

Todo pedido comienza como PENDIENTE. La opción de completar cambia su estado a COMPLETO y la eliminación lo cambia a ELIMINADO. La lista actual incluye pendientes y completos, mientras que los eliminados se excluyen de búsquedas y conteos actuales. El filtro por estado y el contador de pendientes consultan el mismo conjunto de datos.

Al completar o eliminar se añade al historial una copia del pedido con el estado de ese momento. Si un pedido completo se elimina después, el historial conserva las dos transiciones. Se impide completar dos veces un pedido y modificar el nombre de un pedido servido. Estas reglas se aplican en el modelo.

Correcciones del código inicial

Problema encontrado	**Corrección**
Buscar por tipo comparaba el nombre del plato.	Se compara getTipo con el tipo solicitado.
Un índice inválido seguía llegando a remove.	El ID se valida y se rechaza antes de modificar datos.

Problema encontrado	Corrección
La opción de buscar no tenía break.	Cada operación termina su caso sin ejecutar el siguiente.
El do while terminaba con la opción de eliminar.	El programa termina exclusivamente con 0 o al finalizar la entrada.
Faltaban actualización, estados, historial y Main.	Se implementaron las funciones y la clase de inicio.
El modelo imprimía errores y podía devolver null.	El modelo devuelve datos o una excepción; la vista muestra los mensajes.

Explicación de los archivos de la actividad

Archivo Pedido

lab6_actividad1/src/modelo/Pedido.java

Representa un pedido individual. Contiene id, nombrePlato, tipo y estado. El constructor recibe el identificador, el nombre y el tipo; limpia los espacios externos y establece PENDIENTE como estado inicial. Los getters permiten consultar el pedido. setNombrePlato cambia el nombre y setEstado registra una transición.

El ID no tiene setter, de modo que permanece estable. El tipo se mantiene después del registro porque la actividad solicita actualizar el nombre. Esta clase no imprime mensajes ni lee el teclado. Las reglas para modificar o completar un pedido se comprueban en PedidoModelo antes de llamar a sus setters.

Archivo EstadoPedido

lab6_actividad1/src/modelo/EstadoPedido.java

Define los tres valores permitidos: PENDIENTE, COMPLETO y ELIMINADO. Usé un enum para evitar escribir el estado como una cadena que pudiera contener errores de ortografía o combinaciones no previstas. Pedido guarda un valor de esta enumeración y PedidoModelo lo utiliza al filtrar registros, validar cambios y contar pendientes.

Archivo PedidoModelo

lab6_actividad1/src/modelo/PedidoModelo.java

Administra dos listas: pedidos conserva los registros y su estado actual; historial conserva copias de las transiciones. siguienteId comienza en 1 y aumenta después de un registro válido. agregarPedido valida nombre y tipo. buscarPorId recorre la lista y rechaza IDs inexistentes o eliminados. eliminarPedido cambia el estado y registrarHistorial crea la copia que conserva el resultado.

actualizarPedido comprueba que el pedido siga pendiente y que el nuevo nombre tenga contenido. buscarPedido y buscarPedidoTipo recorren getPedidos y comparan el campo correspondiente. contarPedidos construye un TreeMap con un contador por tipo; cuando ya existe una clave aumenta su valor y, en caso contrario, la inicia en 1.

completarPedido valida el estado anterior, cambia a COMPLETO y registra el evento. pedidosPorEstado devuelve una lista filtrada; contarPendientes usa su tamaño. getPedidos construye la lista de registros no eliminados y getHistorial devuelve otra lista con las copias guardadas. El modelo no depende de la vista ni del controlador, y no contiene System.out.

Archivo PedidoVista

lab6_actividad1/src/vista/PedidoVista.java

Se encarga de la consola. Scanner lee líneas completas, lo que evita mezclar nextInt con nextLine. solicitarTexto presenta un mensaje y devuelve la entrada, o null cuando termina el flujo. solicitarNumero convierte una línea con Integer.parseInt y transforma un error de formato en un mensaje comprensible.

mostrarPedidos presenta ID, plato, tipo y estado, o indica que la lista está vacía. mostrarConteo recibe el total y el mapa por tipo, sin calcular reglas de negocio. mostrarMenu y mostrarMenuBuscar enumeran las opciones. solicitarOpcion lee la selección y mostrarMensaje imprime las respuestas del controlador. cerrarScanner libera el lector al salir. Los métodos solicitarNombrePlato y solicitarTipoPlato conservan las entradas específicas del ejemplo de la guía.

Archivo PedidoControlador

lab6_actividad1/src/controlador/PedidoControlador.java

Recibe PedidoModelo y PedidoVista mediante el constructor. iniciar mantiene un do while con un switch, solicita los datos de cada operación, llama al modelo y presenta el resultado. El método auxiliar solicitarTexto detecta entradas incompletas; las excepciones de validación se capturan para que el menú continúe.

buscarPedidos selecciona nombre o tipo y pasa a la vista la lista que devuelve el modelo. mostrarPorEstado permite escoger pendientes o completos. Los casos de registrar, eliminar, actualizar, contar y completar llaman a los métodos del modelo; los casos de pendientes e historial muestran sus resultados. Cada caso tiene break y 0 termina el ciclo. Al finalizar se cierra el Scanner de la vista.

Archivo Main de pedidos

lab6_actividad1/src/principal/Main.java

Contiene el punto de entrada main. Instancia PedidoModelo, después PedidoVista y finalmente PedidoControlador con esas dos referencias. Luego llama a iniciar. Separar el arranque permite localizar rápidamente qué objetos se necesitan y evita colocar el menú o las reglas de pedidos en la clase principal.

Archivo del módulo de pedidos

lab6_actividad1/src/module-info.java

Declara el módulo laboratorio.pedidos. No necesita requires adicionales porque las colecciones y Scanner pertenecen a java.base, disponible de forma implícita. El nombre del módulo utiliza palabras sin dígitos finales para evitar advertencias del compilador. La ejecución por classpath indicada en el anexo también funciona.

Opciones de la actividad

Opción	Resultado
1 y 2	Registrar un pedido y mostrar los pedidos actuales.
3 y 4	Eliminar por ID y buscar por nombre o tipo.
5 y 6	Actualizar el nombre y contar total y por tipo.
7 y 8	Completar un pedido y mostrar pendientes o completos.
9 y 10	Contar pendientes y mostrar el historial.
0	Finalizar el programa.

Imágenes de ejecución de la actividad

Registro y listado

Se registran Sopa como Entrada y Pollo como Fondo. El listado permite comprobar que cada registro conserva su ID y comienza pendiente.

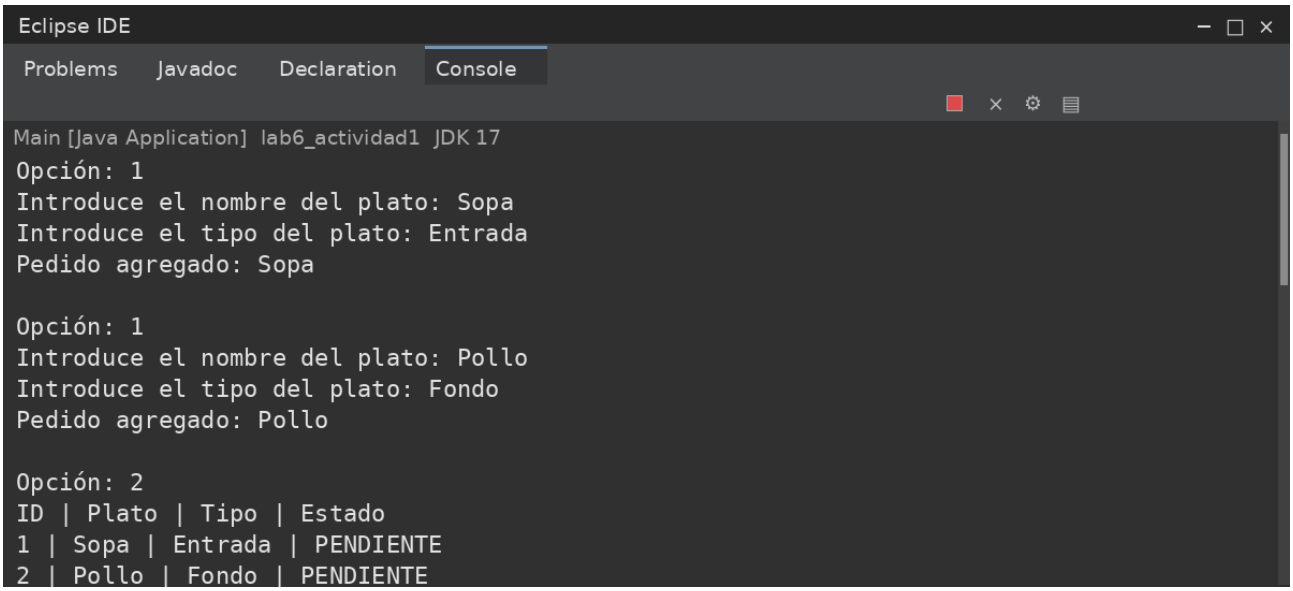


Figura 1. Registro y listado. Fuente: elaboración propia.

Actualización búsqueda y conteo

Se cambia el nombre del pedido 2 a Pollo al horno. La búsqueda por fondo encuentra su tipo aunque se escriba en minúsculas. El conteo indica dos pedidos actuales y uno de cada tipo.

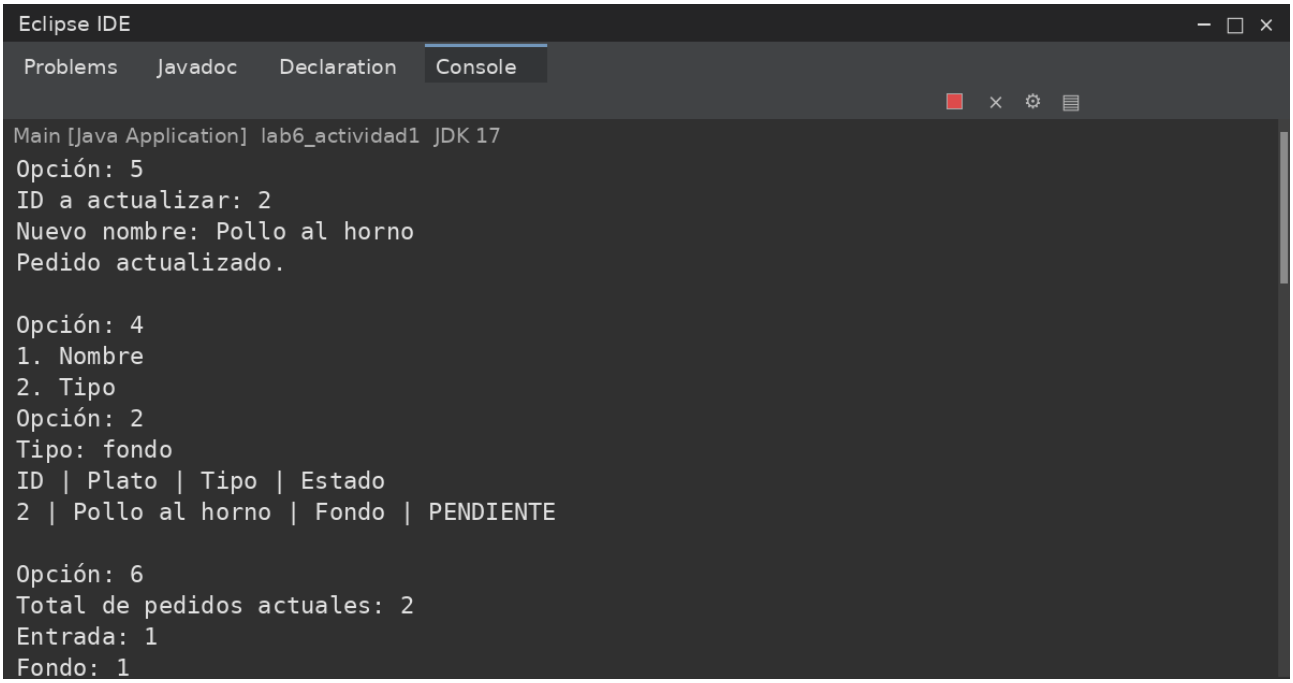


Figura 2. Actualización búsqueda y conteo. Fuente: elaboración propia.

Estados pendientes e historial

El pedido 1 se completa y el pedido 2 se elimina. El filtro muestra la sopa servida, el contador indica un pendiente antes de eliminar el pollo, y el historial conserva el resultado de ambas operaciones.

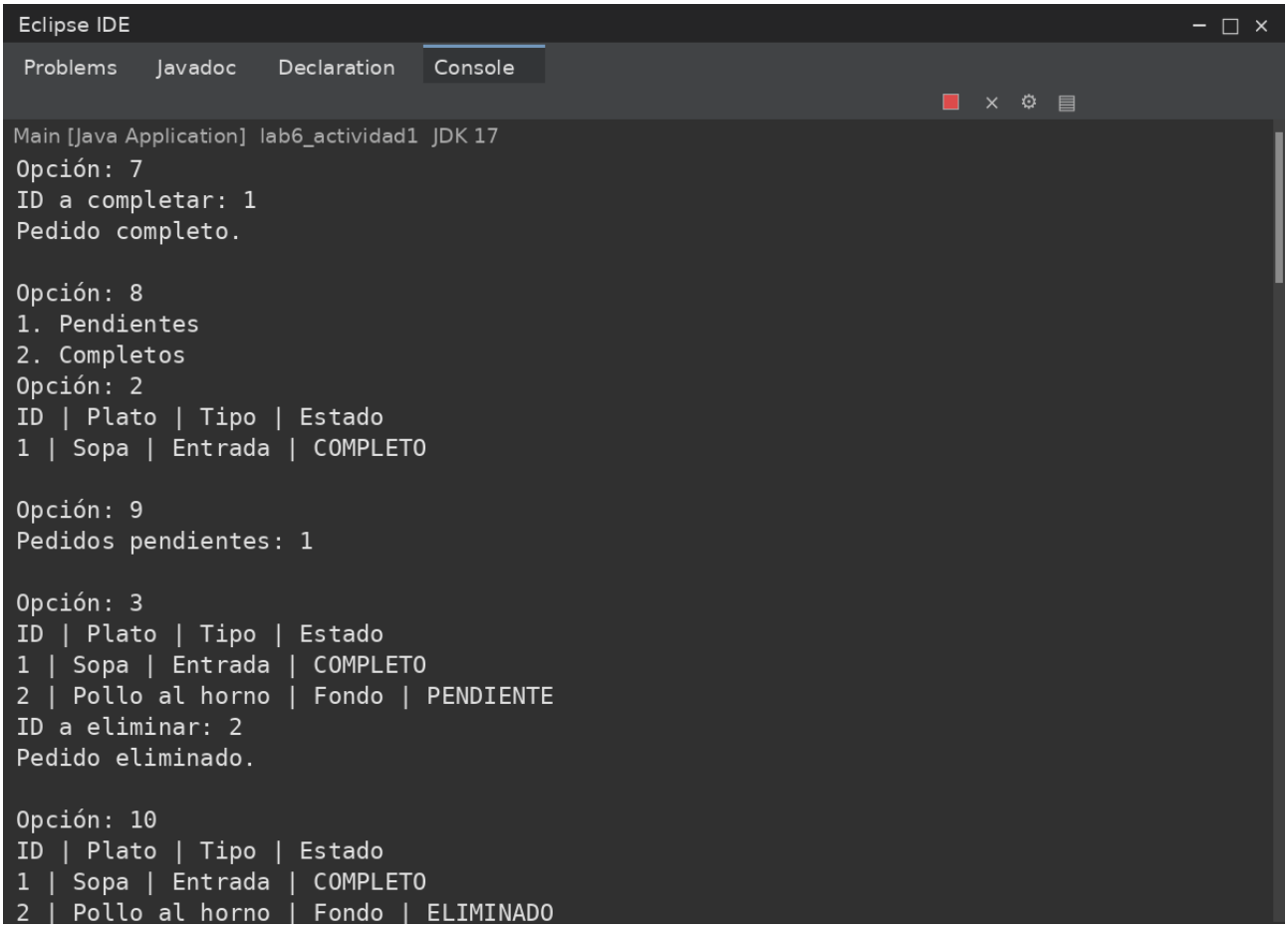


Figura 3. Estados pendientes e historial. Fuente: elaboración propia.

Validación de entradas

Un ID inexistente y una entrada que no es un número generan mensajes claros. El programa sigue funcionando y no modifica los datos debido al error.

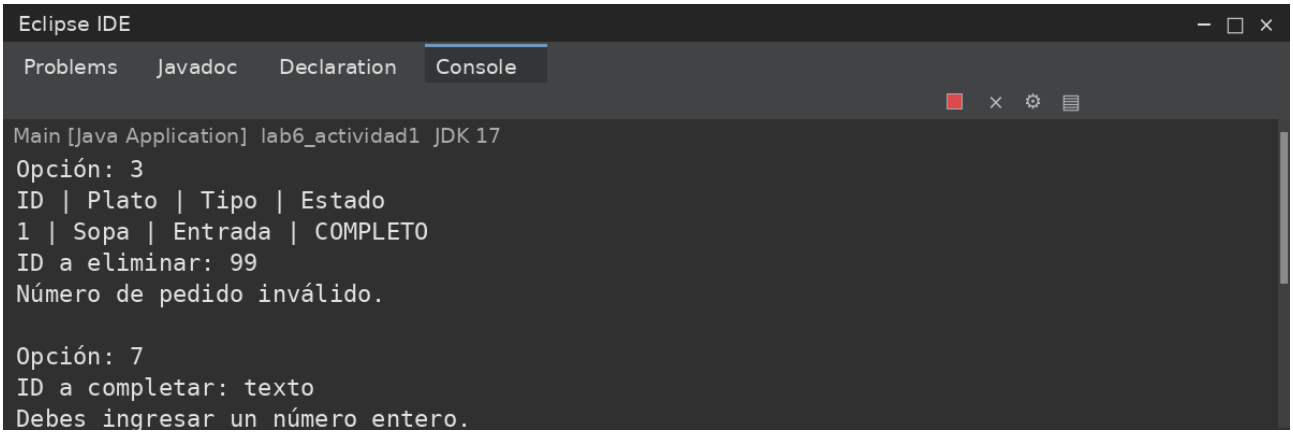


Figura 4. Validación de entradas. Fuente: elaboración propia.

3 Ejercicios propuestos

Ejercicio 1 Carrito de compras

El sistema permite registrar productos, listarlos, agregar cantidades al carrito, revisar su contenido, retirar un producto, aplicar un descuento porcentual, elegir una modalidad de envío, confirmar la compra y consultar el historial. Al agregar otra vez el mismo producto se acumulan sus unidades en una sola línea.

El dinero se maneja con BigDecimal. El precio admite hasta dos decimales; se rechazan precios no positivos o superiores a S/ 1 000 000.00. La cantidad válida está entre 1 y 1000 unidades por producto. Estas restricciones evitan valores de entrada sin sentido para la práctica.

Cálculo	**Regla utilizada**
Subtotal	Suma de precio por cantidad de cada producto.
Descuento	Subtotal por porcentaje dividido entre 100; redondeo HALF_UP a dos decimales.
Envío	RECOJO S/ 0.00, LOCAL S/ 10.00, NACIONAL S/ 25.00.
Total	Subtotal menos descuento más envío.
Compra	Requiere sesión, carrito con productos y confirmación S.

La confirmación N conserva el carrito. La confirmación S crea una Compra, copia sus líneas y guarda subtotal, descuento y envío de esa operación. Luego se vacía el carrito y se restablecen descuento y envío. Por eso una compra anterior no cambia cuando se arma otro carrito.

Ejercicio 2 Autenticación reseñas y calificaciones

El registro solicita nombre, correo y contraseña. El correo se normaliza a minúsculas y no puede repetirse. Se comprueba un formato básico de correo y una contraseña de al menos ocho caracteres. Iniciar sesión exige que el correo y la contraseña coincidan con un usuario registrado.

No se guarda la contraseña original dentro de Usuario. La clase genera una sal aleatoria de 16 bytes y obtiene un hash de 256 bits con PBKDF2WithHmacSHA256 y 120 000 iteraciones. Al iniciar sesión se vuelve a derivar el hash con la misma sal y se compara con MessageDigest.isEqual. Esta implementación sirve para la autenticación local del laboratorio.

Cada compra se asocia al ID del usuario. El historial solo muestra las compras de la sesión actual. Para reseñar, el modelo comprueba que exista una compra confirmada de ese usuario que contenga el producto. La calificación debe ser de 1 a 5, el texto debe tener contenido y se permite una reseña por usuario y producto. Las reseñas pueden consultarse junto con su promedio.

La sesión se exige antes de manejar el carrito. Para entrar con otra cuenta se debe cerrar la sesión anterior; al cerrarla se limpian el carrito, el descuento y el envío. Esto evita que otro usuario vea un

carrito pendiente de la cuenta anterior. El catálogo y las reseñas publicadas pueden consultarse sin sesión.

Explicación de los archivos de los ejercicios

Archivo Producto

lab6_ejercicios1_2/src/modelo/Producto.java

Representa un artículo del catálogo mediante id, nombre y precio. El constructor recibe los tres valores y los getters permiten consultarlos. No contiene setters, por lo que el precio que se usa en una línea del carrito no cambia por una edición posterior del catálogo.

El precio es BigDecimal para mantener aritmética decimal en los importes. TiendaModelo valida y ajusta el precio antes de crear Producto. Esta clase conserva los datos del artículo; no conoce usuarios, menús ni compras.

Archivo ItemCarrito

lab6_ejercicios1_2/src/modelo/ItemCarrito.java

Agrupa un Producto con su cantidad. Los getters permiten mostrar la línea y setCantidad permite acumular unidades de un producto existente. getSubtotal multiplica el precio por BigDecimal.valueOf(cantidad).

Usé una clase para la línea porque el producto del catálogo y la cantidad seleccionada tienen responsabilidades diferentes. Así se pueden conservar dos teclados en una compra y uno en un carrito nuevo sin modificar el Producto.

Archivo Compra

lab6_ejercicios1_2/src/modelo/Compra.java

Guarda id, usuarioId, fecha, productos, subtotal, descuento y envío. El constructor toma LocalDateTime.now y crea un ItemCarrito nuevo por cada línea recibida. Esa copia evita que cambiar la cantidad del carrito modifique una compra ya confirmada.

getTotal usa los importes guardados al comprar y calcula subtotal menos descuento más envío. getProductos devuelve una nueva lista para que retirar una línea de la lista retornada no retire registros de la compra. Los getters entregan el resto de datos a la vista. No se vuelve a calcular el historial con el descuento del carrito actual.

Archivo Usuario

lab6_ejercicios1_2/src/modelo/Usuario.java

Representa una cuenta con id, nombre, correo, sal y claveHash. El constructor crea la sal con SecureRandom y llama a generarHash. Ese método usa PBEKeySpec, SecretKeyFactory y PBKDF2WithHmacSHA256; en finally limpia la contraseña temporal del objeto PBEKeySpec.

verificarClave recibe la contraseña escrita, deriva su hash y lo compara con el almacenado. Los getters exponen únicamente id, nombre y correo. No existe un getter para la contraseña, la sal o el hash. El modelo de usuarios decide si una cuenta puede registrarse o iniciar sesión; Usuario realiza la comprobación de la clave.

Archivo UsuarioModelo

lab6_ejercicios1_2/src/modelo/UsuarioModelo.java

Administra la lista de cuentas y la referencia usuarioActual. registrar limpia nombre y correo, normaliza el correo con Locale.ROOT, valida sus datos y recorre la lista para rechazar duplicados antes de crear Usuario. Los IDs se obtienen del número de cuentas porque este sistema no elimina usuarios.

iniciarSesion busca el correo normalizado y verifica la contraseña; solo si ambos son correctos establece usuarioActual. Una credencial incorrecta genera una excepción. cerrarSesion asigna null. getUsuarioActual permite consultar la sesión y exigirSesion devuelve el usuario o rechaza una operación protegida. El modelo no imprime información ni maneja el carrito.

Archivo Resena

lab6_ejercicios1_2/src/modelo/Resena.java

Conserva productoId, usuarioId, nombreUsuario, calificacion y comentario. El constructor obtiene la identidad del Usuario que reseña. Esto relaciona la reseña con la cuenta y permite mostrar el nombre del comprador sin recibirlo como texto libre desde la consola.

La clase contiene getters y no modifica compras ni sesiones. TiendaModelo verifica previamente la compra, el rango de calificación y la ausencia de una reseña anterior. Luego crea Resena con el texto limpio.

Archivo TiendaModelo

lab6_ejercicios1_2/src/modelo/TiendaModelo.java

Centraliza el catálogo, el carrito, las compras y las reseñas en cuatro ArrayList. También guarda porcentajeDescuento y zonaEnvio. agregarProducto valida nombre y precio y asigna un ID. buscarProducto recorre el catálogo y rechaza un ID inexistente. agregarAlCarrito valida cantidad, acumula unidades si la línea existe y crea una línea nueva en caso contrario. eliminarDelCarrito busca por ID y retira la línea completa.

aplicarDescuento acepta porcentajes entre 0 y 100 y reemplaza el anterior, sin acumular descuentos. seleccionarEnvio normaliza y valida la zona. calcularSubtotal suma los subtotales; calcularDescuento aplica el porcentaje y el redondeo; calcularEnvio devuelve la tarifa elegida o cero si no hay productos; calcularTotal reúne los tres importes.

realizarCompra verifica usuario y carrito, construye la compra con importes y líneas actuales, la añade a compras y llama a vaciarCarrito. Este último reinicia también descuento y envío.

historialUsuario filtra compras por usuarioId. productoComprado recorre ese historial y las líneas de cada compra para comprobar que el producto realmente se compró.

agregarResena busca el producto, verifica al comprador, valida texto y calificación y rechaza la combinación repetida de usuario y producto. resenasProducto devuelve las reseñas del artículo y promedioProducto divide la suma entre la cantidad, devolviendo cero si no existen reseñas. getProductos y getCarrito retornan listas nuevas. Ninguna función del modelo lee el teclado ni imprime mensajes.

Archivo TiendaVista

lab6_ejercicios1_2/src/vista/TiendaVista.java

Recibe un Scanner desde Main. solicitarTexto lee una línea, solicitarNumero convierte un entero y solicitarPrecio crea un BigDecimal aceptando punto o coma decimal. solicitarOpcion devuelve 0 cuando termina la entrada para cerrar el menú. Los errores de formato se expresan mediante excepciones que el controlador muestra.

mostrarMenu enumera las 14 operaciones y la salida. mostrarProductos presenta ID, nombre y precio; mostrarItems presenta cantidad y subtotal por línea. mostrarTotales recibe importes ya calculados. mostrarCompra muestra ID, fecha, líneas e importes, y mostrarHistorial aplica esa presentación a todas las compras recibidas. mostrarResenas presenta autor, nota, texto y promedio, formateado con dos decimales. La vista decide cómo se ve la información y no quién puede comprar o reseñar.

Archivo UsuarioVista

lab6_ejercicios1_2/src/vista/UsuarioVista.java

Comparte el Scanner de TiendaVista para evitar que dos lectores distintos consuman System.in. solicitarTexto presenta los campos de registro y acceso. mostrarUsuario indica si hay sesión y, cuando existe, muestra el nombre. mostrarMensaje presenta confirmaciones.

Separé esta vista para que UsuarioControlador no tenga que manejar la presentación de productos ni las compras. La contraseña se solicita por consola dentro de la práctica; Scanner no oculta los caracteres escritos. Los ejemplos de ejecución usan credenciales ficticias.

Archivo UsuarioControlador

lab6_ejercicios1_2/src/controlador/UsuarioControlador.java

Coordina UsuarioModelo y UsuarioVista. registrar solicita nombre, correo y clave y llama al modelo. iniciarSesion solicita credenciales y muestra la confirmación únicamente después del acceso correcto. cerrarSesion elimina la sesión en el modelo y presenta el resultado. mostrarUsuario delega la presentación del usuario actual.

La validación del correo y la clave permanece en UsuarioModelo y Usuario. Las excepciones llegan a TiendaControlador, que las muestra dentro del menú general. Esto mantiene las operaciones de autenticación separadas de las operaciones comerciales.

Archivo TiendaControlador

lab6_ejercicios1_2/src/controlador/TiendaControlador.java

Recibe TiendaModelo, TiendaVista, UsuarioModelo y UsuarioControlador. iniciar usa un do while y un switch para coordinar las 14 opciones. Antes de manejar un carrito, comprar, ver historial o reseñar llama a exigirSesion. El registro y acceso se delegan a UsuarioControlador; después de un acceso correcto se limpia el carrito.

mostrarCarrito entrega a la vista las líneas y los importes calculados por el modelo. comprar valida sesión y contenido, muestra el total y solicita S o N. Solo S llama a realizarCompra y muestra el comprobante local. Al cerrar sesión se llama a vaciarCarrito. La opción de reseñar solicita ID, nota y texto; la decisión de permitirla se realiza en TiendaModelo.

Los errores de entrada o negocio se capturan como IllegalArgumentException. ArithmeticException cubre los precios con decimales no admitidos y muestra un mensaje específico. La compra cancelada no registra una compra ni vacía el carrito. Los casos incluyen break y 0 finaliza el ciclo.

Archivo Main de la tienda

lab6_ejercicios1_2/src/principal/Main.java

Crea un solo Scanner, los dos modelos y las dos vistas. Después construye UsuarioControlador y lo entrega a TiendaControlador junto con sus demás dependencias. Llama a iniciar y cierra el Scanner al terminar.

El constructor de cada componente recibe los objetos que necesita, sin utilizar variables globales ni Singleton. Ambas vistas reciben la misma entrada de consola. La clase principal permite ver la composición de las dos partes MVC antes de iniciar la interacción.

Archivo del módulo de la tienda

lab6_ejercicios1_2/src/module-info.java

Declara laboratorio.tienda. Las colecciones, BigDecimal, LocalDateTime y las clases criptográficas utilizadas forman parte de java.base. Por eso el módulo no requiere bibliotecas externas. El código puede compilarse con JDK 17 y ejecutarse desde Eclipse o mediante los comandos del anexo.

Opciones de los ejercicios

Opción	Operación
1 y 2	Agregar y listar productos.
3 a 5	Agregar, ver y eliminar productos del carrito.
6 y 7	Aplicar descuento y seleccionar envío.
8 y 9	Confirmar compra y consultar el historial propio.

Opción	Operación
10 a 12	Registrar cuenta, iniciar sesión y cerrar sesión.
13 y 14	Añadir una reseña y consultar reseñas con promedio.
0	Salir.

Imágenes de ejecución del ejercicio 1

Catálogo de productos

Se agregan un teclado de S/ 120.00 y un mouse de S/ 50.00. El catálogo muestra sus IDs y precios con dos decimales.

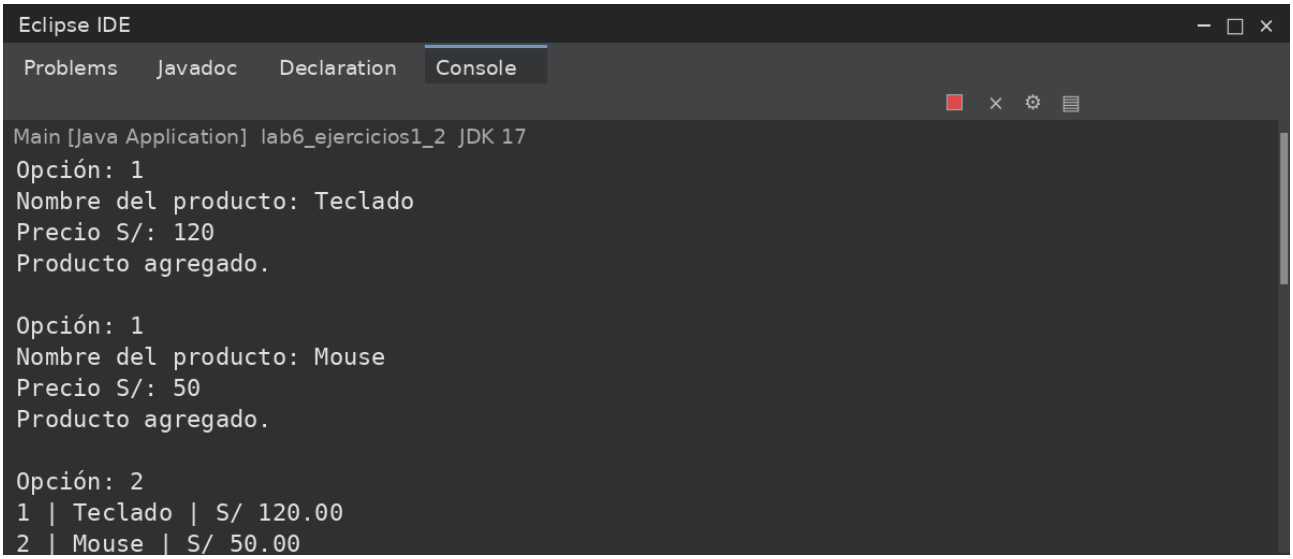


Figura 5. Catálogo de productos. Fuente: elaboración propia.

Carrito descuento y envío

Se prueba retirar el mouse y volver a añadirlo. El carrito contiene dos teclados y un mouse, suma S/ 290.00, descuenta S/ 29.00 y agrega S/ 10.00 de envío local. El total es S/ 271.00.

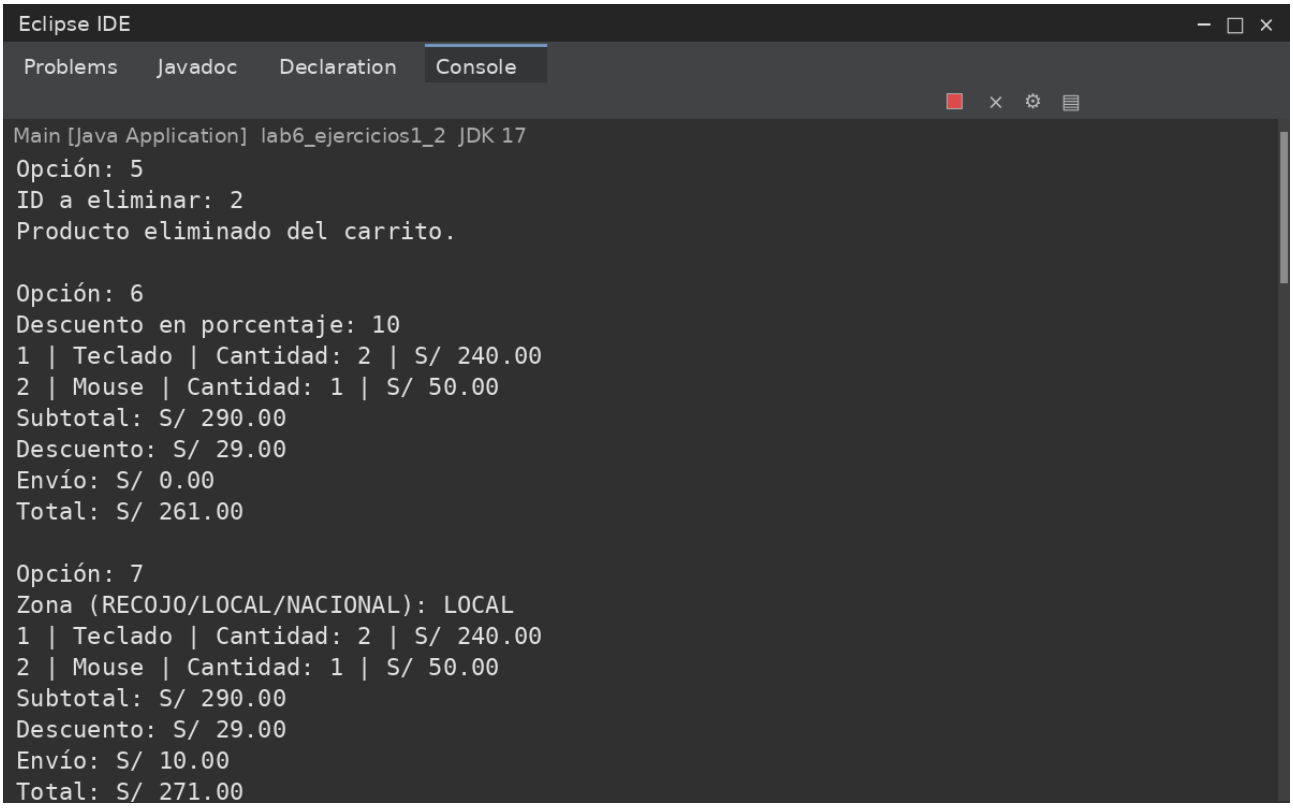


Figura 6. Carrito descuento y envío. Fuente: elaboración propia.

Compra confirmada e historial

La confirmación S registra la compra. Consultar el historial devuelve las mismas líneas y los mismos importes guardados, incluso después de vaciarse el carrito.

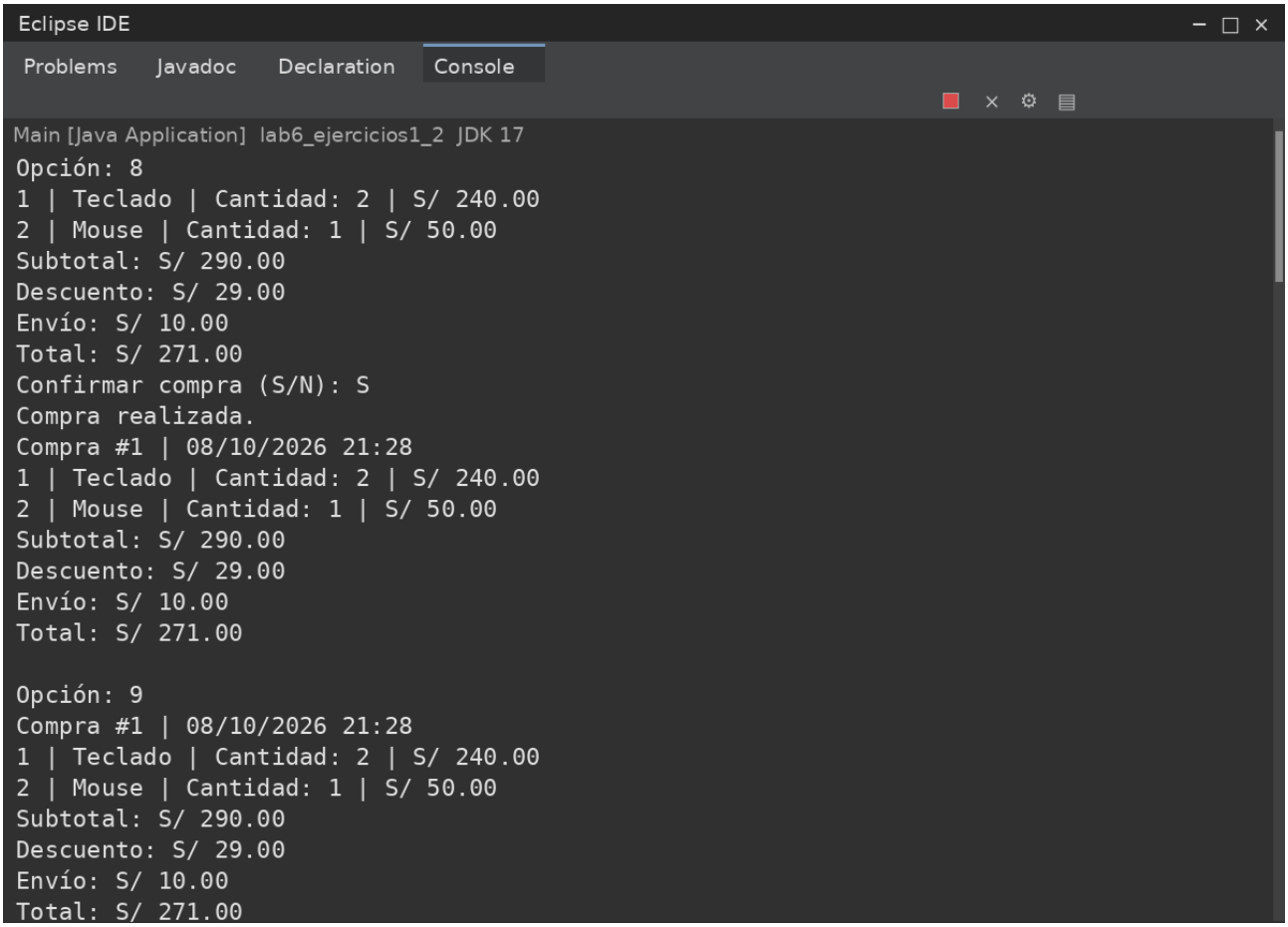


Figura 7. Compra confirmada e historial. Fuente: elaboración propia.

Imágenes de ejecución del ejercicio 2

Registro e inicio de sesión

Se registra la cuenta de prueba, se intenta entrar con una clave incorrecta y luego se accede con la clave válida. El mensaje de sesión iniciada aparece únicamente en el intento correcto.

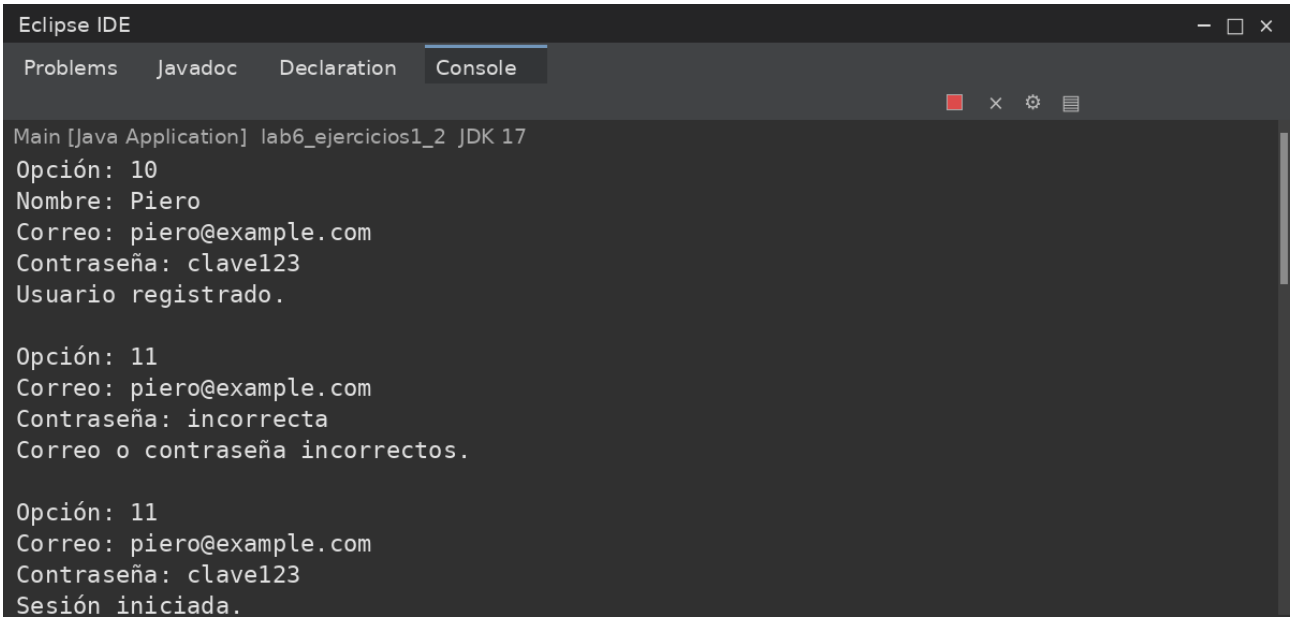


Figura 8. Registro e inicio de sesión. Fuente: elaboración propia.

Reseña calificación y promedio

El comprador califica el teclado con 5 y deja una reseña. La consulta muestra el autor y el promedio 5.00 sobre 5. Un segundo intento de reseña del mismo usuario y producto es rechazado.

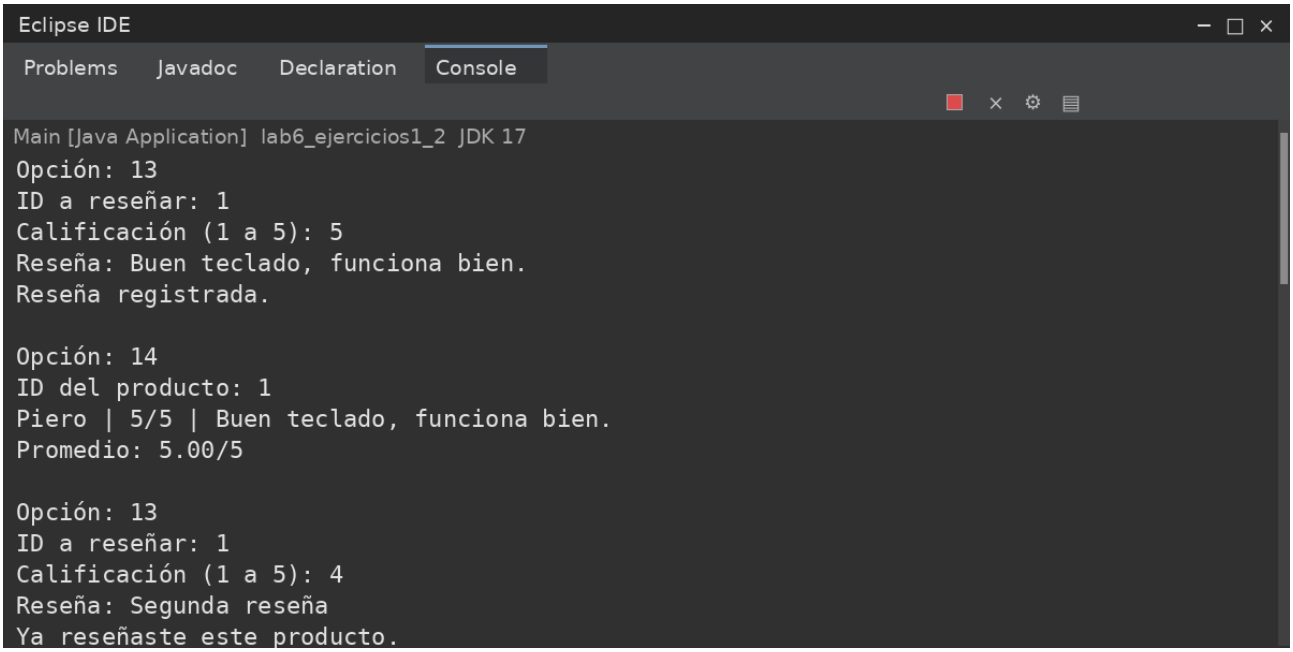


Figura 9. Reseña calificación y promedio. Fuente: elaboración propia.

Control de compras por usuario

Después de iniciar sesión con Ana, el historial no muestra la compra de Piero. Ana tampoco puede calificar ese producto porque su cuenta no ha realizado una compra que lo contenga.

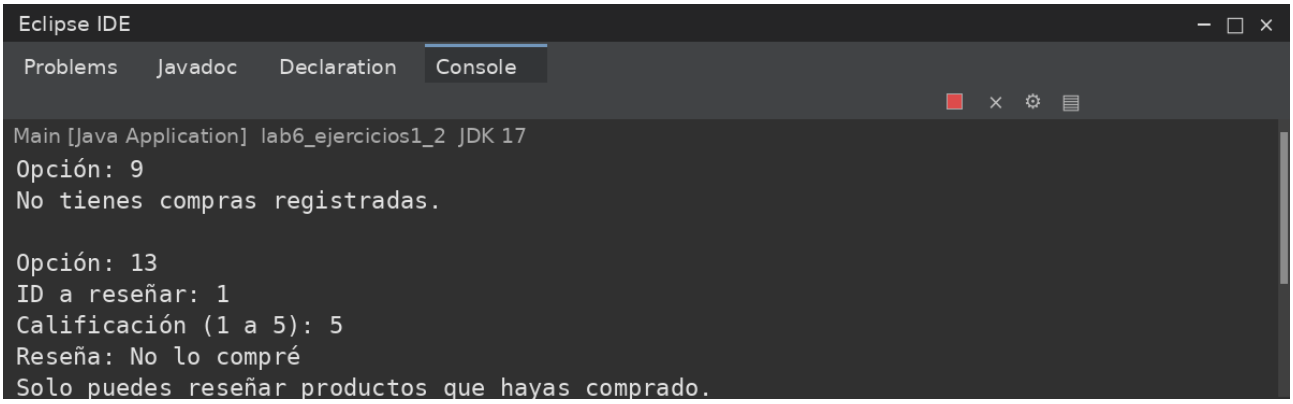


Figura 10. Control de compras por usuario. Fuente: elaboración propia.

Verificación de resultados

Los dos proyectos se compilaron con Java 17. Verificacion ejecutó 53 comprobaciones sobre registros, búsquedas, IDs, estados, historial, autenticación, cantidades, precios, descuentos, envío y reseñas. También se ejecutaron sesiones interactivas que probaron actualización, entradas inválidas, cancelación de compra, compra confirmada y cambio de usuario. Los resultados de consola se encuentran en pruebas/resultados.txt.

Estas pruebas comprueban el comportamiento implementado y los casos de entrada ensayados. El programa no incorpora persistencia, disponibilidad multiusuario simultánea ni un procesador de pagos; su alcance corresponde a una aplicación local de laboratorio.

4 Cuestionario

Pregunta 1

¿Qué es un patrón arquitectónico y cuál es su propósito en el desarrollo de software?

Es una solución reutilizable que organiza los componentes y sus relaciones para un tipo de problema de software. Sirve para orientar decisiones de estructura, comunicación y distribución de responsabilidades. Perry y Wolf (1992) explican la arquitectura mediante elementos, forma y justificación; Gamma et al. (1993) muestran cómo los patrones capturan experiencia de diseño. Un patrón ofrece una base adaptable, mientras que el proyecto debe justificar cómo la aplica.

Pregunta 2

¿Cuáles son algunos ejemplos de patrones arquitectónicos populares?

Entre los ejemplos se encuentran capas, cliente servidor, tuberías y filtros, MVC y microservicios. Las capas separan responsabilidades por nivel; cliente servidor distribuye solicitudes y servicios; tuberías y filtros encadena transformaciones; MVC separa dominio, presentación e interacción. Los microservicios organizan capacidades en servicios con autonomía de despliegue. Los estilos estructurales se describen en Perry y Wolf (1992), MVC en Krasner y Pope (1988), y la evolución y dificultades de microservicios en Jamshidi et al. (2018).

Pregunta 3

¿Cómo se diferencia un patrón arquitectónico de una arquitectura de software?

El patrón describe una organización reutilizable; la arquitectura corresponde a las decisiones concretas de un sistema. Indicar que una aplicación usa MVC no define todas sus clases, conectores, persistencia ni despliegue. La arquitectura debe precisar esos elementos y justificar sus restricciones. Por ejemplo, en esta práctica MVC se concreta en modelos en memoria, vistas de consola y controladores con menús. La distinción entre una arquitectura específica y sus estilos se fundamenta en Perry y Wolf (1992).

Pregunta 4

¿Qué consideraciones se deben tener en cuenta al seleccionar un patrón arquitectónico para un proyecto?

Se deben considerar los casos de uso, la complejidad de la interfaz, las cualidades requeridas, la tecnología disponible y el costo de mantener la solución. Syromiatnikov y Weyns (2014) relacionan la elección de los patrones de interfaz con requisitos de calidad y tecnología. También se evalúan despliegue, independencia de equipos y comunicación: Jamshidi et al. (2018) muestran que la autonomía de microservicios incorpora dificultades distribuidas. Para esta práctica, una aplicación local pequeña justifica un MVC de consola.

Pregunta 5

¿Cómo interactúan entre sí el Modelo, la Vista y el Controlador en el patrón MVC?

El usuario realiza una acción; el controlador interpreta esa entrada y solicita una operación al modelo. El modelo aplica las reglas y cambia o consulta su estado. La vista muestra el resultado. En MVC clásico puede existir notificación de cambios para que varias vistas se actualicen, como describen Krasner y Pope (1988). En este laboratorio el controlador obtiene el resultado y lo pasa a la vista de forma explícita. El modelo no necesita conocer la consola.

Pregunta 6

¿Cuáles son las ventajas de utilizar el patrón MVC en aplicaciones web?

Separar la presentación de la lógica del dominio facilita modificar una interfaz, reutilizar operaciones y probar responsabilidades por separado. Leff y Rayfield (2001) estudian MVC en aplicaciones web y proponen mantener la lógica independiente de decisiones de partición entre cliente y servidor. Singh et al. (2018) analizan Spring MVC y Struts 2 para facilitar el desarrollo de aplicaciones Java. Estas ventajas dependen de mantener la separación: colocar toda la lógica en un controlador reduce su utilidad.

Pregunta 7

¿En qué situaciones podría no ser recomendable usar el patrón MVC?

Puede resultar innecesario para un programa pequeño sin interacción de usuario, como una transformación de archivos ejecutada por lotes. En interfaces con enlaces intensivos entre datos y componentes, otra variante puede expresar mejor la presentación. Syromiatnikov y Weyns (2014) indican que la elección debe atender casos de uso y tecnología, y Fuksa et al. (2025) documentan variantes y compromisos de MVVM. Aplicado a esta pregunta, la conclusión es que dividir artificialmente cualquier programa en tres partes no garantiza un mejor diseño.

Pregunta 8

¿Cómo se puede implementar el patrón MVC en aplicaciones de escritorio?

Se crean clases del dominio independientes de la interfaz, una vista que presenta componentes y un controlador que conecta las acciones con operaciones del modelo. La separación de responsabilidades se fundamenta en Krasner y Pope (1988). Como aplicación de ese principio a Java, con Swing los botones y tablas pueden pertenecer a la vista y sus acciones llamar al controlador; con JavaFX se puede separar la vista FXML de las clases del dominio. Este laboratorio utiliza consola para practicar el mismo flujo sin añadir una interfaz gráfica.

Pregunta 9

¿Qué otros patrones similares a MVC existen y en qué se diferencian en el contexto de Java?

MVP utiliza un presentador para coordinar la presentación y las acciones de una vista, lo que permite sustituirla en pruebas. MVVM incorpora un ViewModel que expone datos y operaciones de presentación y suele utilizar enlaces de datos. Syromiatnikov y Weyns (2014) comparan estas familias; Fuksa et al. (2025) precisan que MVVM tiene distintas variantes y compromisos. En Java, una interfaz Swing puede organizarse con un presentador y una interfaz JavaFX puede aprovechar propiedades y bindings. Son decisiones de implementación, no restricciones del lenguaje.

Pregunta 10

¿Cómo se pueden combinar patrones de diseño con arquitectónicos en un proyecto Java?

El patrón arquitectónico organiza el sistema y los patrones de diseño resuelven colaboraciones locales dentro de esa organización. Gamma et al. (1993) describen los patrones como estructuras reutilizables que contribuyen a una arquitectura mayor. Por ejemplo, dentro de MVC, Observer puede notificar cambios del modelo, Strategy puede intercambiar reglas de descuento y Factory Method puede organizar la creación de objetos. Se incorporan cuando existe una necesidad concreta; en este carrito se mantuvo un cálculo directo porque la guía solo exige un descuento porcentual.

Pregunta 11

¿Qué herramientas y frameworks Java facilitan la implementación de patrones de diseño y arquitectónicos?

Spring MVC y Struts 2 ofrecen infraestructura para organizar el manejo de solicitudes y la presentación en aplicaciones web Java; Singh et al. (2018) analizan ambos al formular un framework MVC. Como aplicación práctica a escritorio, Swing y JavaFX permiten construir vistas y gestionar acciones. Eclipse facilita organizar paquetes, compilar, depurar y ejecutar proyectos. Un IDE apoya el trabajo, pero la separación arquitectónica depende de las responsabilidades del código. En esta entrega se usan Eclipse y la biblioteca estándar, sin dependencias externas.

5 Bibliografía

Las referencias corresponden a artículos de revista y artículos publicados en actas científicas. Se verificaron los datos de publicación con los registros de los autores, editoriales y Consensus cuando estuvo disponible. No se incluyen libros de texto, blogs ni tutoriales en esta bibliografía.

Fuksa, M., Speth, S., & Becker, S. (2025). MVVM revisited: Exploring design variants of the Model-View-ViewModel pattern. En J. Borbinha, T. P. Sales, M. M. da Silva, H. A. Proper, & M. Schnellmann (Eds.), *Enterprise design, operations, and computing* (pp. 163-181). Springer. https://doi.org/10.1007/978-3-031-78338-8_9

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1993). Design patterns: Abstraction and reuse of object-oriented design. En O. M. Nierstrasz (Ed.), *ECOOP 93 Object-oriented programming* (pp. 406-431). Springer. https://doi.org/10.1007/3-540-47910-4_21

Jamshidi, P., Pahl, C., Mendonça, N. C., Lewis, J., & Tilkov, S. (2018). Microservices: The journey so far and challenges ahead. *IEEE Software, 35*(3), 24-35. https://doi.org/10.1109/MS.2018.2141039

Krasner, G. E., & Pope, S. T. (1988). A cookbook for using the model-view controller user interface paradigm in Smalltalk-80. *Journal of Object-Oriented Programming, 1*(3), 26-49. https://ics.uci.edu/~dfredmil/ics227-SQ04/papers/KrasnerPope88.pdf

Leff, A., & Rayfield, J. T. (2001). Web-application development using the Model/View/Controller design pattern. *Proceedings Fifth IEEE International Enterprise Distributed Object Computing Conference*, 118-127. https://doi.org/10.1109/EDOC.2001.950428

Perry, D. E., & Wolf, A. L. (1992). Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes, 17*(4), 40-52. https://doi.org/10.1145/141874.141884

Singh, A., Chawla, P., Singh, K., & Singh, A. K. (2018). Formulating an MVC framework for web development in JAVA. *2018 2nd International Conference on Trends in Electronics and Informatics ICOEI*, 926-929. https://doi.org/10.1109/ICOEI.2018.8553746

Syromiatnikov, A., & Weyns, D. (2014). A journey through the land of Model-View-design patterns. *2014 IEEE/IFIP Conference on Software Architecture*, 21-30. https://doi.org/10.1109/WICSA.2014.13

6 Anexos

Anexo A Repositorio y código fuente

Repositorio de la práctica: https://github.com/Crudo27/Lab6

La entrega conserva el proyecto de pedidos del repositorio y añade el proyecto de los ejercicios 1 y 2. Los enlaces de las explicaciones llevan directamente al archivo correspondiente. El informe y las diez imágenes se encuentran en docs, junto con los archivos fuente y resultados de verificación en sus carpetas respectivas.

Archivos de las actividades

lab6_actividad1/src/controlador/PedidoControlador.java

lab6_actividad1/src/modelo/EstadoPedido.java

lab6_actividad1/src/modelo/Pedido.java

lab6_actividad1/src/modelo/PedidoModelo.java

lab6_actividad1/src/module-info.java

lab6_actividad1/src/principal/Main.java

lab6_actividad1/src/vista/PedidoVista.java

Archivos de los ejercicios

lab6_ejercicios1_2/src/controlador/TiendaControlador.java

lab6_ejercicios1_2/src/controlador/UsuarioControlador.java

lab6_ejercicios1_2/src/modelo/Compra.java

lab6_ejercicios1_2/src/modelo/ItemCarrito.java

lab6_ejercicios1_2/src/modelo/Producto.java

lab6_ejercicios1_2/src/modelo/Resena.java

lab6_ejercicios1_2/src/modelo/TiendaModelo.java

lab6_ejercicios1_2/src/modelo/Usuario.java

lab6_ejercicios1_2/src/modelo/UsuarioModelo.java

lab6_ejercicios1_2/src/module-info.java

lab6_ejercicios1_2/src/principal/Main.java

lab6_ejercicios1_2/src/vista/TiendaVista.java

lab6_ejercicios1_2/src/vista/UsuarioVista.java

Anexo B Archivos de configuración y verificación

Archivo	Uso en la entrega
.project	Existe en cada proyecto. Identifica su nombre y configura el constructor Java de Eclipse.

Archivo	Uso en la entrega
.classpath	Existe en cada proyecto. Define src como entrada, bin como salida y JavaSE-17 como entorno.
org.eclipse.core.resources.prefs	Configura UTF-8 para que los textos en español se lean correctamente.
org.eclipse.jdt.core.prefs	Define nivel de fuente, compatibilidad y destino Java 17 y desactiva características preview.
fuentes.txt	Existe en cada proyecto. Enumera todos sus archivos Java para compilar con @fuentes.txt.
.gitignore	Excluye clases compiladas, bin y metadatos temporales del IDE.
README.md	Explica alcance, ejecución, reglas del sistema y ubicación de los entregables.
Verificacion.java	Ejecuta 53 comprobaciones de los modelos. comprobar verifica una condición y rechazar exige una excepción ante datos inválidos.
resultados.txt	Conserva la salida obtenida al ejecutar Verificacion para revisar cada resultado.
docs/capturas/*.png	Contiene diez imágenes de consola, descritas individualmente en las figuras 1 a 10.
Informe PDF y Word	Contienen la explicación de archivos, decisiones, cuestionario, bibliografía y anexos de esta práctica.

Anexo C Compilación y ejecución

En Eclipse se importa cada carpeta como proyecto Java existente y se ejecuta principal.Main. Si no se detecta JavaSE-17, se selecciona un JDK instalado de versión 17 o superior. Los dos proyectos tienen su propio Main y deben ejecutarse por separado.

Desde la carpeta de cada proyecto, los comandos son:

```
javac -encoding UTF-8 --release 17 -d bin @fuentes.txt
java -cp bin principal.Main
```

En el entorno utilizado para verificar la entrega se invocó el mismo compilador mediante java com.sun.tools.javac.Main porque estaba disponible el módulo jdk.compiler y no el lanzador javac. Se usaron UTF-8 y --release 17 en ambos proyectos.

Después de compilar los proyectos, desde la raíz del repositorio en Linux o macOS:

```
javac -encoding UTF-8 --release 17
-cp lab6_actividad1/bin:lab6_ejercicios1_2/bin
-d pruebas/bin pruebas/Verificacion.java
```

Las tres líneas anteriores forman un solo comando; se unen con espacios al copiarlo en la terminal.

```
java -cp lab6_actividad1/bin:lab6_ejercicios1_2/bin:pruebas/bin Verificacion
```

En Windows se utiliza ; en lugar de : como separador del classpath y se encierra esa ruta entre comillas. La última línea del resultado esperado es: Resultado: 53 verificaciones correctas.

Anexo D Correspondencia con la rúbrica

Criterio técnico	**Evidencia presentada**
Completitud	Actividades 1, 2 y 3; pareja de ejercicios 1 y 2; once respuestas y bibliografía.
Modelo	Entidades, colecciones, validaciones, cálculos, estados y autenticación.
Vista	Menús, entradas y mensajes concentrados en las clases vista.
Controlador	Operaciones coordinadas mediante switch, validaciones de sesión y manejo de errores.
Interacción	Entradas de la vista, operaciones del modelo y presentación del resultado.
Documentación y código	Explicación por archivo, nombres descriptivos, paquetes MVC e imágenes; código sin comentarios por indicación de entrega.